字节跳动三面：通信稳定性与算子基准

> 技术深挖复盘 · CUDA Graph / TP / 融合通信 / MoE / Benchmark。根据个人转写改写，公司与轮次沿用来源文件标注，不代表官方题库。

一、这一轮主要考什么

核心是稳定性问题的定位过程，以及算子开发与模型、硬件知识的连接。故障修复不仅要描述“改了什么”，还需要解释版本、数据类型、同步机制和最小回归。

二、源记录中可确认的提问

1. Graph 开启且 TP 大于一时出现 hang，怎样设计开关和卡数对照？
2. 如何将问题进一步缩小到某个融合 All-Reduce / 归一化算子？
3. 数据完成标记放在哪里、由谁写入、怎样轮询和更新？
4. 为什么原实现使用浮点载荷或特殊值作为完成标记？设计边界是什么？
5. 位模式判断、数据类型和等待条件之间怎样对应？
6. 修复后做了哪些两卡和模型级 Graph 开关回归？
7. 为什么选择 CUDA、Triton 或 CuTe DSL？如何考虑可维护性和协作？
8. MoE 推训路由不一致具体表现在哪里，路由回放记录和复用什么？
9. 一个 token 通过 MoE 的前向流程是什么？有哪些计算与通信步骤？
10. 确定性 dQ 路径的瓶颈、分块和依赖怎样解释？
11. 算子 benchmark 怎样组织 shape、dtype、batch 和多种实现？
12. 怎样输出可复用的测试结果，而非手工汇报一个加速倍数？
13. 评价某种 GPU 时，哪些结论来自规格、哪些来自自己的实测？
14. 下一步怎样补硬件细节和通信方面的理解？

三、整理补充：稳定性案例的完整结构

从现象收敛到路径

制作实验矩阵，列出 Graph 开关、TP 卡数、目标融合路径开关、输入类型、版本和是否复现。每次只改变明确因素，保留“没有复现”对应的测试时长和次数。不能把一次成功运行当成根因已经证明。

从路径收敛到协议

画两侧写入、轮询、完成和复用缓冲区的状态图。将载荷位宽、哨兵位模式、比较宽度和状态发布顺序逐一对应到代码。原记录存在负零、十六位/三十二位与常量的识别歧义，本篇不根据口述给出具体十六进制修复值。

从补丁走到回归

先构造能触发等待错误的最小用例，再做不同类型、卡数、Graph 模式和重复回放的回归；明确修复的是哪个项目版本或内部派生版本。不要把某个库在特定调用链中的融合路径，描述成 CUDA Graph 通用地自动融合算子。

四、算子基准如何准备

准备统一输入表，包含 shape、layout、dtype、随机种子、误差标准、预热和计时范围。分别比较手写实现、计算库实现与原框架路径，记录版本及选中算法。表中每一项收益都能回到具体用例，而非只给平均值。

硬件比较还应固定软件、工作负载与计时方法。对未经公开规格或同口径测试核对的制程、缓存层级、带宽及相对性能，只列为待研究问题，不写成产品评价结论。

五、来源与范围

来源：`字节 AI INFRA 三面 (transcribed on 14-Sep-2026 22-12-45).txt`。原文件部分文字以单字空格分隔，已按上下文整理。删除身份、同事姓名、内部组织和模型信息、薪酬与流程；未收录未核实的硬件对比结论。与已有综合复盘不同，本篇集中补充通信故障收敛与基准测试的连续追问。